

Rapport Final : Smart Agentic Medical Research Assistant

1. Introduction

Ce projet présente le développement d'un assistant de recherche médicale de pointe, conçu pour répondre aux besoins de précision et de fiabilité du domaine de la santé. L'innovation majeure réside dans son architecture agentic capable de choisir dynamiquement entre plusieurs sources de données : une base de connaissances locale (RAG) et des bases de données mondiales (PubMed, FDA).

2. Architecture du Système

L'architecture repose sur un moteur de décision central (Agent) orchestrant trois modules principaux, comme détaillé dans le tableau ci-dessous :

Composant	Technologie	Rôle
Cerveau (Agent)	LangChain + GPT-4o-mini	Analyse l'intention de l'utilisateur et choisit l'outil optimal pour répondre à la requête.
Mémoire (RAG)	FAISS + OpenAI Embeddings	Permet la recherche et l'extraction d'informations pertinentes à partir des documents PDF fournis par l'utilisateur.
Recherche (APIs)	PubMed & OpenFDA	Offre un accès en temps réel à la littérature scientifique via PubMed et aux données réglementaires sur les médicaments via OpenFDA.

3. Fonctionnement Technique

3.1. Le Moteur de Décision

L'agent utilise le paradigme **ReAct** (Reasoning and Acting) pour son fonctionnement. Lorsqu'une question est posée par l'utilisateur, l'agent génère une pensée interne pour déterminer si la réponse peut être trouvée dans les documents téléchargés et indexés localement, ou s'il est nécessaire d'interroger une API externe pour obtenir des informations à jour ou spécifiques [1]. Ce processus garantit une réponse contextualisée et pertinente.

3.2. Le Pipeline RAG

Le processus de Retrieval Augmented Generation (RAG) suit plusieurs étapes clés pour construire et interroger la base de connaissances locale. Premièrement, l'**ingestion** des documents PDF est réalisée via la bibliothèque `PyPDF`. Ensuite, le texte est fragmenté en **chunks** de 1000 caractères avec un chevauchement de 100 caractères pour préserver le contexte lors de la récupération. Ces fragments sont ensuite convertis en **vecteurs numériques** à l'aide du modèle d'embeddings `text-embedding-3-small` d'OpenAI. Enfin, ces vecteurs sont stockés et indexés dans une base de données vectorielle `FAISS`, permettant une recherche rapide et efficace des informations pertinentes [2].

4. Résultats et Démonstration

L'interface utilisateur, développée avec **Streamlit**, offre une interaction fluide et intuitive avec l'assistant. Grâce à cette interface, l'utilisateur peut effectuer diverses actions :

- **Télécharger une étude clinique** ou tout autre document PDF pertinent, puis poser des questions spécifiques sur son contenu. L'agent utilisera alors le module RAG pour extraire les informations pertinentes.
- **Demander les derniers articles** sur une pathologie ou un sujet médical précis. L'agent interrogera l'API PubMed pour récupérer les publications les plus récentes et pertinentes.

- **Vérifier les indications, contre-indications ou effets secondaires** d'un médicament. L'agent utilisera l'API OpenFDA pour fournir des informations officielles et détaillées.

Chaque réponse est accompagnée des sources utilisées, assurant la transparence et la fiabilité des informations fournies.

5. Conclusion

Ce projet démontre la puissance et la polyvalence des agents IA hybrides dans le domaine médical. En combinant le RAG pour la personnalisation et l'accès aux connaissances locales, et les APIs pour l'actualité et les données réglementaires, nous avons développé un outil robuste, évolutif et professionnel. Cet assistant est parfaitement adapté aux exigences du secteur médical, offrant une aide précieuse aux professionnels de santé et aux étudiants pour leurs recherches et analyses [3].

6. Références

[1] LangChain. (n.d.). *Agents*. <https://www.langchain.com/> [2] OpenAI. (n.d.). *Embeddings*. <https://openai.com/> [3] Streamlit. (n.d.). *Streamlit Documentation*. <https://docs.streamlit.io/>